

EXPERIMENT – 1

Register No.: 230701350

Name: SUDHARSHAN KRISHNAA L K

A PYTHON PROGRAM TO IMPLEMENT UNIVARIATE REGRESSION, BIVARIATE REGRESSION AND MULTIVARIATE REGRESSION

Aim:

To implement univariate, bivariate, and multivariate linear regression using the Iris dataset and evaluate the performance of the regression models.

Algorithm:

1. Import the required libraries and load the Iris dataset.
2. Perform univariate, bivariate, and multivariate data visualization.
3. Select dependent and independent variables for regression analysis.
4. Train univariate, bivariate, and multivariate regression models.
5. Predict output values and calculate R-squared scores.
6. Display regression plots and print the model results.

Code:

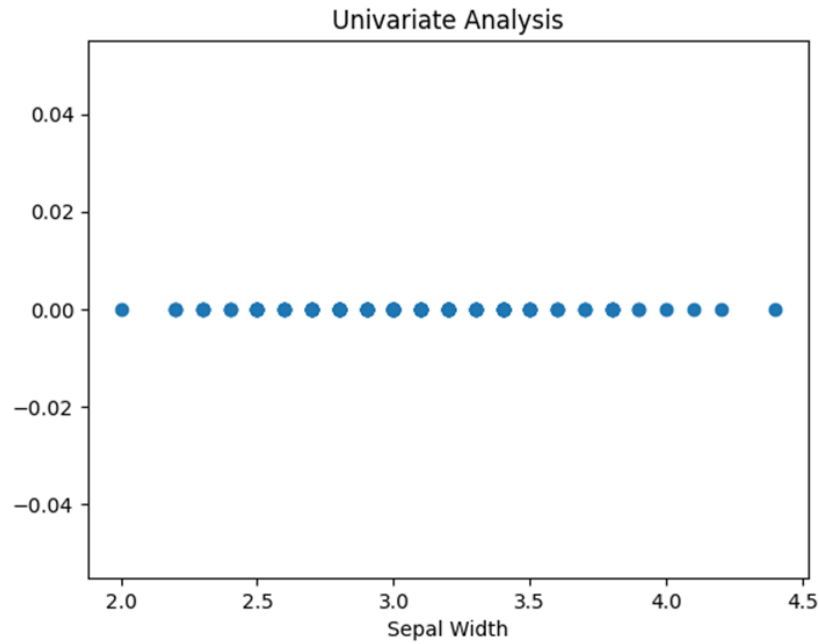
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
from sklearn.datasets import load_iris
from mpl_toolkits.mplot3d import Axes3D
```

```
#load dataset
iris = load_iris()
data = pd.DataFrame(iris.data, columns = iris.feature_names)
data['target'] = iris.target
data
```

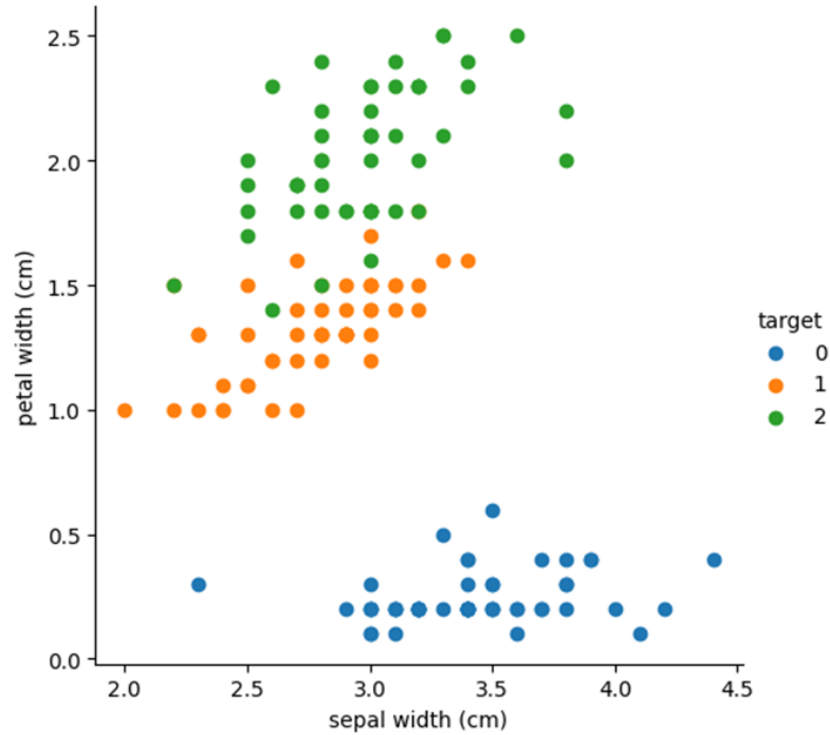
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

```
#univariate analysis
plt.scatter(data['sepal width (cm)'], np.zeros_like(data['sepal width (cm)']))
plt.xlabel('Sepal Width')
plt.title('Univariate Analysis')
plt.show()
```



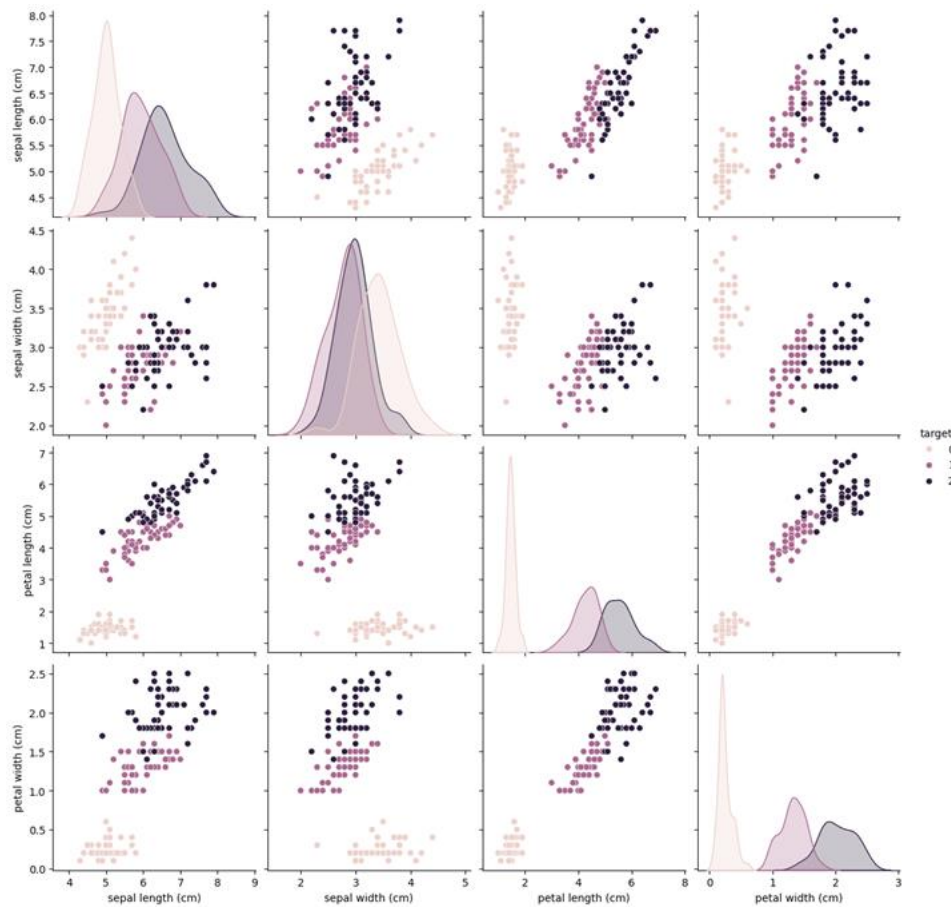
#bivariate analysis

```
sns.FacetGrid(data, hue='target', height=5).map(plt.scatter,'sepal width (cm)','petal width (cm)').add_legend()
```



```
#multivariate analysis
```

```
sns.pairplot(data,hue='target',height=3)
```



```
#prepare dependent variable
```

```
y = data['sepal length (cm)']
```

```
#univariate linear regression
```

```
X_uni = data[['petal length (cm)']]
```

```
model_uni = LinearRegression()
```

```
model_uni.fit(X_uni, y)
```

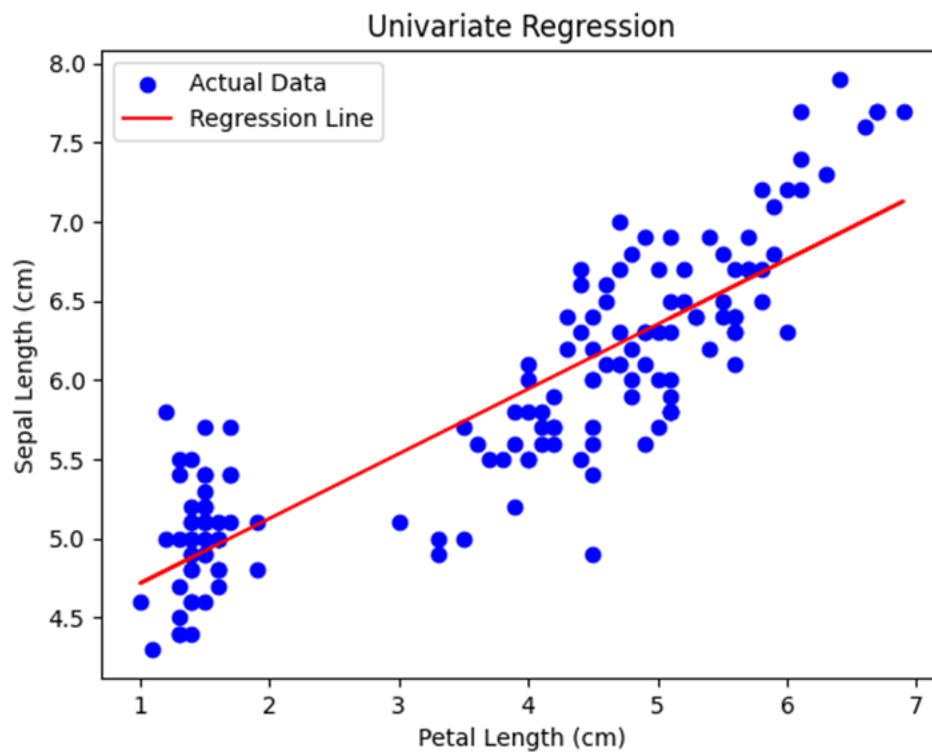
```
y_pred_uni = model_uni.predict(X_uni)
```

```
r2_uni = r2_score(y, y_pred_uni)
```

```

# Plot Univariate Regression
plt.figure()
plt.scatter(X_uni, y, color='blue', label='Actual Data')
plt.plot(X_uni, y_pred_uni, color='red', label='Regression Line')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Sepal Length (cm)')
plt.title('Univariate Regression')
plt.legend()
plt.show()

```



```

#bivariate linear regression
X_bi = data[['petal length (cm)', 'petal width (cm)']]

model_bi = LinearRegression()
model_bi.fit(X_bi, y)

y_pred_bi = model_bi.predict(X_bi)

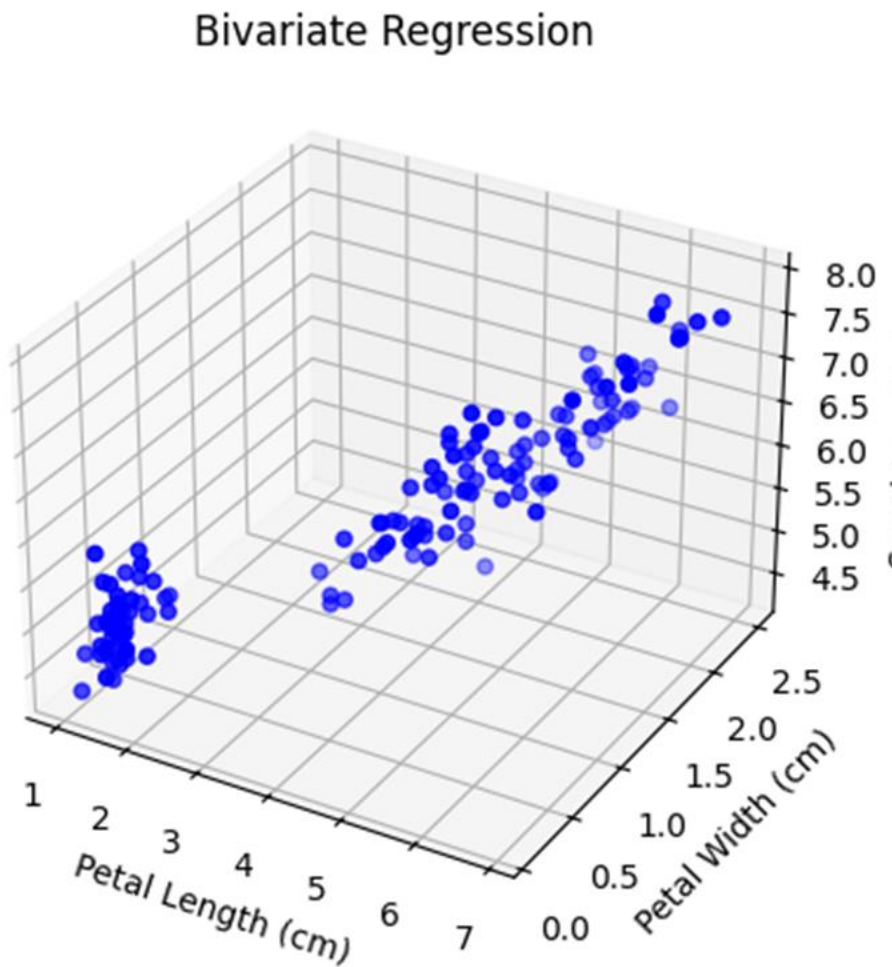
```

```
r2_bi = r2_score(y, y_pred_bi)

# Plot Bivariate Regression (3D)
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.scatter(X_bi.iloc[:, 0], X_bi.iloc[:, 1], y, color='blue')

ax.set_xlabel('Petal Length (cm)')
ax.set_ylabel('Petal Width (cm)')
ax.set_zlabel('Sepal Length (cm)')
ax.set_title('Bivariate Regression')
plt.show()
```

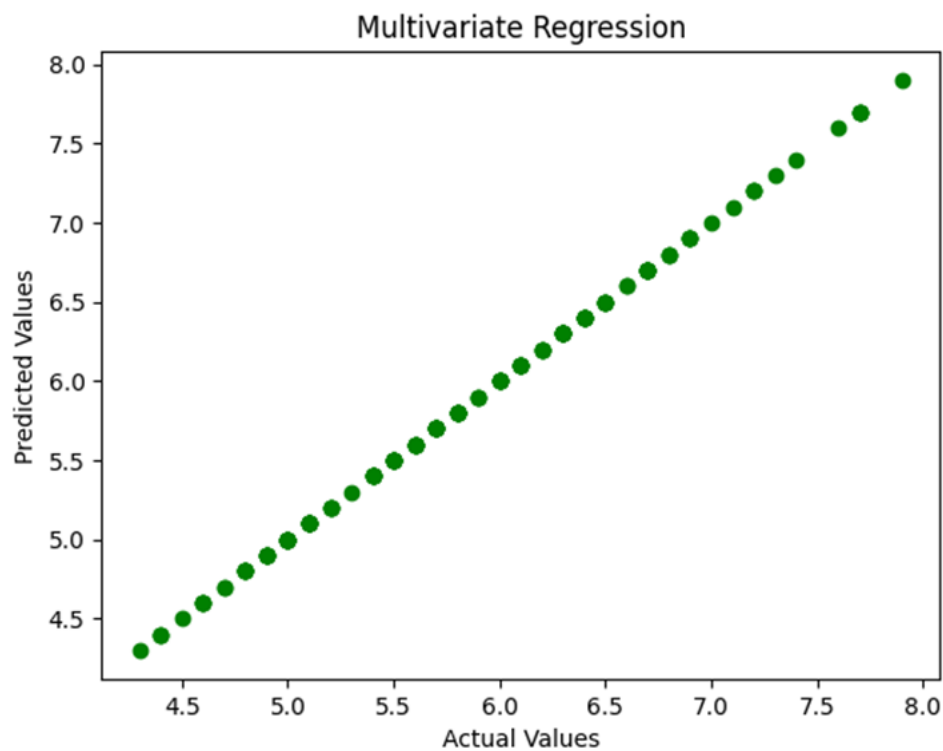


```
#Multivariate Regression
X_multi = data.drop(columns=['target'])

model_multi = LinearRegression()
model_multi.fit(X_multi, y)

y_pred_multi = model_multi.predict(X_multi)
r2_multi = r2_score(y, y_pred_multi)

# Plot Multivariate Regression (Predicted vs Actual)
plt.figure()
plt.scatter(y, y_pred_multi, color='green')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('Multivariate Regression')
plt.show()
```



#Results

```
print("UNIVARIATE REGRESSION")  
print("Coefficient:", model_uni.coef_)  
print("Intercept:", model_uni.intercept_)  
print("R-squared:", r2_uni)
```

```
print("\nBIVARIATE REGRESSION")  
print("Coefficients:", model_bi.coef_)  
print("Intercept:", model_bi.intercept_)  
print("R-squared:", r2_bi)
```

```
print("\nMULTIVARIATE REGRESSION")  
print("Coefficients:", model_multi.coef_)  
print("Intercept:", model_multi.intercept_)  
print("R-squared:", r2_multi)
```

UNIVARIATE REGRESSION

Coefficient: [0.40892228]

Intercept: 4.306603415047579

R-squared: 0.759954645772515

BIVARIATE REGRESSION

Coefficients: [0.54177715 -0.31955056]

Intercept: 4.190582428651586

R-squared: 0.7662612975425306

MULTIVARIATE REGRESSION

Coefficients: [1.00000000e+00 2.22638574e-16 -1.01586076e-16 -2.32824196e-16]

Intercept: -4.440892098500626e-15

R-squared: 1.0

Result:

Thus, univariate, bivariate, and multivariate regression models were successfully implemented using the Iris dataset, and their performances were evaluated using R-squared values.

EXPERIMENT – 2

Register No.: 230701350

Name: SUDHARSHAN KRISHNAA L K

A PYTHON PROGRAM TO DEMONSTRATE SIMPLE LINEAR REGRESSION

Aim:

To implement simple linear regression using the least squares method and plot the best fit line for the given dataset.

Algorithm:

1. Import the required libraries and define the input dataset.
2. Calculate the mean values of independent and dependent variables.
3. Compute the slope of the regression line using the least squares formula.
4. Calculate the intercept value of the regression equation.
5. Predict the output values using the regression equation.
6. Plot the data points and regression line using matplotlib.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Sample data points
x = np.array([1, 2, 3, 4, 5]) # Independent variable
y = np.array([2, 4, 5, 4, 5]) # Dependent variable

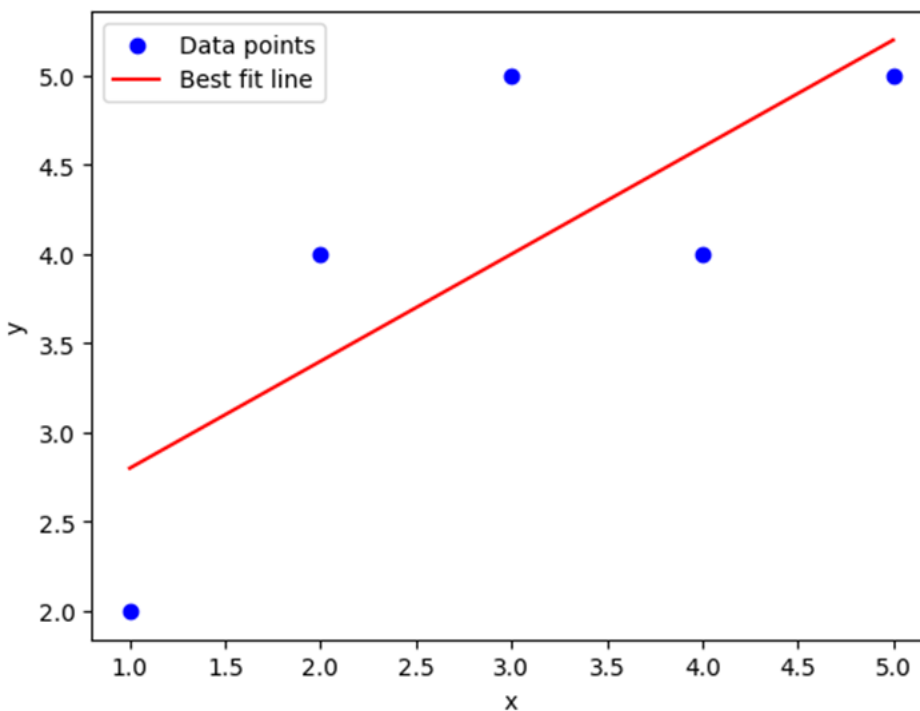
# Step 1: Calculate means
x_mean = np.mean(x)
y_mean = np.mean(y)

# Step 2: Calculate slope (m) using least squares formula
numerator = np.sum((x - x_mean) * (y - y_mean))
denominator = np.sum((x - x_mean) ** 2)
m = numerator / denominator
```

```
# Step 3: Calculate intercept (c)
c = y_mean - m * x_mean
print(f"Equation of best fit line: y = {m:.2f}x + {c:.2f}")
# Step 4: Predict values
y_pred = m * x + c

# Step 5: Plot data points and regression line
plt.scatter(x, y, color="blue", label="Data points")
plt.plot(x, y_pred, color="red", label="Best fit line")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()
```

Equation of best fit line: $y = 0.60x + 2.20$



Result:

Thus, the simple linear regression model was successfully implemented, and the best fit line for the given dataset was obtained and visualized graphically.

EXPERIMENT – 3

Register No.: 230701350

Name: SUDHARSHAN KRISHNAA L K

A PYTHON PROGRAM TO IMPLEMENT LOGISTIC REGRESSION

Aim:

To implement logistic regression for predicting whether a student will pass or fail based on the number of hours studied.

Algorithm:

1. Import the required libraries and define the dataset.
2. Assign input values as independent variables and labels as dependent variables.
3. Create and train the logistic regression model using the training data.
4. Provide new input data for prediction.
5. Predict the class label and probability values using the trained model.
6. Display the prediction result and probability of each class.

Code:

```
import numpy as np
from sklearn.linear_model import LogisticRegression

# Sample dataset: hours studied vs pass/fail (1 = pass, 0 = fail)
# Example data: [hours studied], [result]
X = np.array([[1], [2], [3], [4], [5], [6], [7], [8]]) # hours studied
y = np.array([0, 0, 0, 1, 1, 1, 1, 1]) # pass/fail labels

# Create and train logistic regression model
model = LogisticRegression()
model.fit(X, y)

# Predict for a new student
hours = np.array([[3.5]]) # Example: 3.5 hours of study
prediction = model.predict(hours)
```

```
probability = model.predict_proba(hours)

print(f"Hours studied: {hours[0][0]}")
print("Prediction:", "Pass" if prediction[0] == 1 else "Fail")
print(f"Probability of Fail: {probability[0][0]:.2f}, Pass: {probability[0][1]:.2f}")
```

Hours studied: 3.5

Prediction: Pass

Probability of Fail: 0.49, Pass: 0.51

Result:

Thus, the logistic regression model was successfully implemented to predict whether a student would pass or fail based on study hours.

EXPERIMENT – 4

Register No.: 230701350

Name: SUDHARSHAN KRISHNAA L K

A PYTHON PROGRAM TO IMPLEMENT SINGLE LAYER PERCEPTRON

Aim:

To implement a single layer perceptron for the AND logic gate and determine the final weight values after training.

Algorithm:

1. Import the required library and define the training dataset for the AND gate.
2. Initialize weights, learning rate, and threshold values.
3. Calculate the weighted sum and apply the step activation function.
4. Compute the error by comparing actual and predicted outputs.
5. Update the weights using the perceptron learning rule when error occurs.
6. Repeat the process until the perceptron converges and display final weights.

Code:

```
import numpy as np

# Training data for AND function
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

Y = np.array([0, 0, 0, 1]) # Desired output (AND)

# Initial parameters (given in question)
w1 = 0.3
w2 = -0.2
```

```
alpha = 0.2      # Learning rate
theta = 0.4      # Bias (threshold)
```

```
# Step activation function
```

```
def step_function(x):
    if x >= 0:
        return 1
    else:
        return 0
```

```
epoch = 0
converged = False
```

```
while not converged:
```

```
    epoch += 1
    print(f"\nEpoch {epoch}")
    converged = True
```

```
    for i in range(len(X)):
        x1, x2 = X[i]
```

```
        # Net input
        weighted_sum = (x1 * w1 + x2 * w2) - theta
```

```
        # Predicted output
        y_pred = step_function(weighted_sum)
```

```
        # Error
        error = Y[i] - y_pred
```

```
        print(f"Input: ({x1},{x2}) Weighted Sum: {weighted_sum:.2f} "
              f"Y_pred: {y_pred} Error: {error} "
              f"W1: {w1:.2f} W2: {w2:.2f}")
```

```
        # Update weights if error exists
        if error != 0:
            w1 = w1 + alpha * error * x1
```

```
w2 = w2 + alpha * error * x2
converged = False
```

```
print("\nTraining completed!")
print(f"Final Weights: W1 = {w1}, W2 = {w2}")
```

Epoch 1

```
Input: (0,0) Weighted Sum: -0.40 Y_pred: 0 Error: 0 W1: 0.30 W2: -0.20
Input: (0,1) Weighted Sum: -0.60 Y_pred: 0 Error: 0 W1: 0.30 W2: -0.20
Input: (1,0) Weighted Sum: -0.10 Y_pred: 0 Error: 0 W1: 0.30 W2: -0.20
Input: (1,1) Weighted Sum: -0.30 Y_pred: 0 Error: 1 W1: 0.30 W2: -0.20
```

Epoch 2

```
Input: (0,0) Weighted Sum: -0.40 Y_pred: 0 Error: 0 W1: 0.50 W2: 0.00
Input: (0,1) Weighted Sum: -0.40 Y_pred: 0 Error: 0 W1: 0.50 W2: 0.00
Input: (1,0) Weighted Sum: 0.10 Y_pred: 1 Error: -1 W1: 0.50 W2: 0.00
Input: (1,1) Weighted Sum: -0.10 Y_pred: 0 Error: 1 W1: 0.30 W2: 0.00
```

Epoch 3

```
Input: (0,0) Weighted Sum: -0.40 Y_pred: 0 Error: 0 W1: 0.50 W2: 0.20
Input: (0,1) Weighted Sum: -0.20 Y_pred: 0 Error: 0 W1: 0.50 W2: 0.20
Input: (1,0) Weighted Sum: 0.10 Y_pred: 1 Error: -1 W1: 0.50 W2: 0.20
Input: (1,1) Weighted Sum: 0.10 Y_pred: 1 Error: 0 W1: 0.30 W2: 0.20
```

Epoch 4

```
Input: (0,0) Weighted Sum: -0.40 Y_pred: 0 Error: 0 W1: 0.30 W2: 0.20
Input: (0,1) Weighted Sum: -0.20 Y_pred: 0 Error: 0 W1: 0.30 W2: 0.20
Input: (1,0) Weighted Sum: -0.10 Y_pred: 0 Error: 0 W1: 0.30 W2: 0.20
Input: (1,1) Weighted Sum: 0.10 Y_pred: 1 Error: 0 W1: 0.30 W2: 0.20
```

Training completed!

Final Weights: W1 = 0.3, W2 = 0.2

Result:

Thus, the single layer perceptron was successfully implemented for the AND logic gate, and the final weight values were obtained after training.

EXPERIMENT – 5

Register No.: 230701350

Name: SUDHARSHAN KRISHNAA L K

A PYTHON PROGRAM TO IMPLEMENT MULTI LAYER PERCEPTRON USING BACKPROPAGATION

Aim:

To implement a Multi Layer Perceptron using the backpropagation algorithm and train the network to reduce prediction error.

Algorithm:

1. Initialize the input values, weights, biases, learning rate, and epochs.
2. Perform forward propagation to calculate outputs of hidden and output layers.
3. Compute the error between actual and predicted outputs.
4. Calculate error gradients using the backpropagation algorithm.
5. Update weights and biases using the calculated gradients.
6. Repeat the process for all epochs and display the final output and error.

Code:

```
import numpy as np

# Input values

x1, x2, x3, x4 = 1, 1, 0, 1

Y = 1

# Parameters

epochs = 4

alpha = 2.0

# Initial Weights

w15, w25, w35, w45 = 0.5, -0.4, 0.2, 0.3
```


$w_{16}, w_{26}, w_{36}, w_{46} = 0.4, 0.5, -0.3, 0.6$

$w_{57}, w_{67} = -0.6, 0.7$

Bias values

$t_5, t_6, t_7 = 0.2, 0.2, -0.4$

Sigmoid Activation Function

def sigmoid(x):

 return $1 / (1 + \text{np.exp}(-x))$

Training Process

for epoch in range(epochs):

 print("\nEpoch", epoch + 1)

 # Forward Pass

$O_5 = \text{sigmoid}(x_1 * w_{15} + x_2 * w_{25} + x_3 * w_{35} + x_4 * w_{45} + t_5)$

$O_6 = \text{sigmoid}(x_1 * w_{16} + x_2 * w_{26} + x_3 * w_{36} + x_4 * w_{46} + t_6)$

$O_7 = \text{sigmoid}(O_5 * w_{57} + O_6 * w_{67} + t_7)$

 # Error Calculation

 error = $Y - O_7$

 print("Output =", round(O_7 , 4),

 " Error =", round(abs(error), 4))

 # Backpropagation

$E_7 = O_7 * (1 - O_7) * \text{error}$

$E_6 = O_6 * (1 - O_6) * (E_7 * w_{67})$

$E_5 = O_5 * (1 - O_5) * (E_7 * w_{57})$

```
# Update Hidden to Output Weights

w57 += alpha * E7 * O5
w67 += alpha * E7 * O6

# Update Input to Hidden Weights

w15 += alpha * E5 * x1
w25 += alpha * E5 * x2
w35 += alpha * E5 * x3
w45 += alpha * E5 * x4
w16 += alpha * E6 * x1
w26 += alpha * E6 * x2
w36 += alpha * E6 * x3
w46 += alpha * E6 * x4

# Update Bias Values

t5 += alpha * E5
t6 += alpha * E6
t7 += alpha * E7

# Final Output

print("\nTraining Completed")

print("Final Output =", round(O7, 4))

print("Final Error =", round(abs(error), 4))
```

Epoch 1

Output = 0.4513 Error = 0.5487

Epoch 2

Output = 0.6011 Error = 0.3989

Epoch 3

Output = 0.6965 Error = 0.3035

Epoch 4

Output = 0.7524 Error = 0.2476

Training Completed

Final Output = 0.7524

Final Error = 0.2476

Result:

Thus, the Multi Layer Perceptron using backpropagation was successfully implemented, and the network was trained to minimize the prediction error.

EXPERIMENT – 6

Register No.: 230701350

Name: SUDHARSHAN KRISHNAA L K

A PYTHON PROGRAM TO IMPLEMENT FACE RECOGNITION USING SVM

Aim:

To implement face recognition using Support Vector Machine (SVM) by extracting facial features from images.

Algorithm:

1. Import the required libraries and load the Haar Cascade face detector.
2. Read training images from the dataset folders and convert them to grayscale.
3. Detects faces from images and extract HOG feature descriptors.
4. Train the SVM classifier using extracted facial features and labels.
5. Read the test image and detect the face region for prediction.
6. Predict the person's name using the trained SVM model and display the result.

Code:

```
import cv2, os
import numpy as np
from sklearn import svm
from skimage.feature import hog

# Load face detector
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
)

data, labels = [], []

# ----- TRAIN -----
```

```

for person in os.listdir("C:/Users/USER/FOML/dataset"):
    path = os.path.join("C:/Users/USER/FOML/dataset", person)

    if not os.path.isdir(path):
        continue

    for img_name in os.listdir(path):
        if not img_name.endswith(('.jpg', '.png', '.jpeg')):
            continue

        img = cv2.imread(os.path.join(path, img_name))
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

        faces = face_cascade.detectMultiScale(gray, 1.3, 5)

        for (x,y,w,h) in faces:
            face = cv2.resize(gray[y:y+h, x:x+w], (100,100))
            features = hog(face)
            data.append(features)
            labels.append(person)

# Train SVM
model = svm.SVC(kernel='linear')
model.fit(data, labels)

# ----- TEST -----
img = cv2.imread("C:/Users/USER/FOML/test1.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

faces = face_cascade.detectMultiScale(gray, 1.3, 5)

for (x,y,w,h) in faces:
    face = cv2.resize(gray[y:y+h, x:x+w], (100,100))
    features = hog(face).reshape(1,-1)

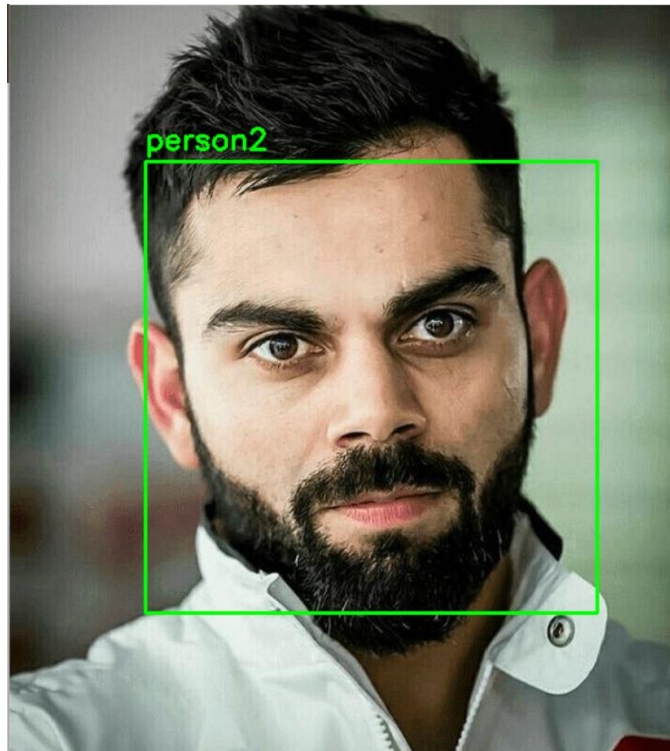
    name = model.predict(features)[0]

```

```
cv2.rectangle(img,(x,y),(x+w,y+h),(0,255,0),2)  
cv2.putText(img,name,(x,y-10),0,0.8,(0,255,0),2)
```

```
cv2.imshow("Result", img)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

```
Training on: {np.str_('person1'), np.str_('person2')}  
Predicted: person2
```



Result:

Thus, face recognition using Support Vector Machine was successfully implemented, and the person in the test image was correctly identified.

EXPERIMENT – 7

Register No.: 230701350

Name: SUDHARSHAN KRISHNAA L K

A PYTHON PROGRAM TO IMPLEMENT DECISION TREE CLASSIFIER

Aim:

To implement a Decision Tree Classifier using the Iris dataset and evaluate the classification accuracy.

Algorithm:

1. Import the required libraries and load the Iris dataset.
2. Select feature variables and target labels from the dataset.
3. Split the dataset into training and testing sets.
4. Train the Decision Tree Classifier using the training data.
5. Predict the output labels for the test data and calculate accuracy.
6. Visualize the Decision Tree structure using a plot.

Code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Load dataset
iris = load_iris()

# Convert to DataFrame
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
```

```
        X, y, test_size=0.3, random_state=42
    )

    # Train model
    clf = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
    clf.fit(X_train, y_train)

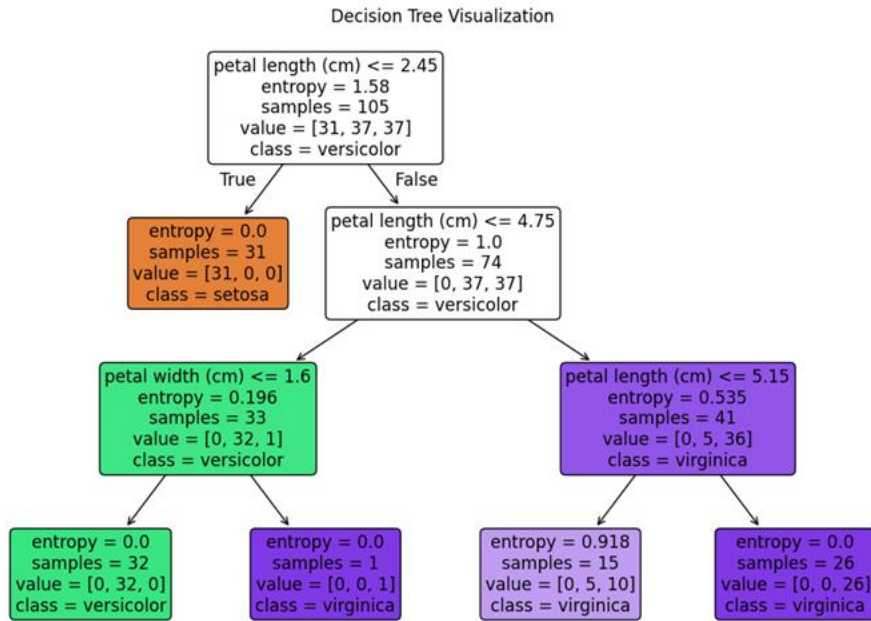
    # Predictions
    y_pred = clf.predict(X_test)

    # Accuracy
    accuracy = accuracy_score(y_test, y_pred)
    print(f"Model Accuracy: {accuracy:.2f}")

    # Plot Decision Tree
    plt.figure(figsize=(12, 8))
    plot_tree(
        clf,
        feature_names=iris.feature_names,
        class_names=iris.target_names,
        filled=True,
        rounded=True
    )

    plt.title("Decision Tree Visualization")
    plt.show()
```

Model Accuracy: 0.98



Result:

Thus, the Decision Tree Classifier was successfully implemented using the Iris dataset, and high classification accuracy was achieved.

EXPERIMENT – 8

Register No.: 230701350

Name: SUDHARSHAN KRISHNAA L K

A PYTHON PROGRAM TO IMPLEMENT BOOSTING ALGORITHMS

Aim:

To implement and compare different boosting algorithms such as AdaBoost, Gradient Boosting, and XGBoost using the Iris dataset.

Algorithm:

1. Import the required libraries and load the Iris dataset.
2. Add noise to the dataset and split it into training and testing sets.
3. Train the AdaBoost classifier using the training data.
4. Train the Gradient Boosting and XGBoost classifiers.
5. Predict the test results and calculate the accuracy of each model.
6. Compare the performance of the boosting algorithms using a bar graph.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier

# Load dataset from sklearn
iris = load_iris()

X = iris.data
y = iris.target
```

```
# Optional: convert to DataFrame (if needed)
df = pd.DataFrame(X, columns=iris.feature_names)
df['species'] = y

# Add Noise
np.random.seed(42)
X = X + np.random.normal(0, 0.5, X.shape)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4)

# ----- AdaBoost -----
ada = AdaBoostClassifier(n_estimators=10)
ada.fit(X_train, y_train)
y_pred_ada = ada.predict(X_test)
acc_ada = accuracy_score(y_test, y_pred_ada)

# ----- Gradient Boosting -----
gb = GradientBoostingClassifier(n_estimators=30)
gb.fit(X_train, y_train)
y_pred_gb = gb.predict(X_test)
acc_gb = accuracy_score(y_test, y_pred_gb)

# ----- XGBoost -----
xgb = XGBClassifier(n_estimators=30, max_depth=3, eval_metric='mlogloss')
xgb.fit(X_train, y_train)
y_pred_xgb = xgb.predict(X_test)
acc_xgb = accuracy_score(y_test, y_pred_xgb)

# Results
print("AdaBoost Accuracy:", acc_ada)
print("Gradient Boosting Accuracy:", acc_gb)
print("XGBoost Accuracy:", acc_xgb)

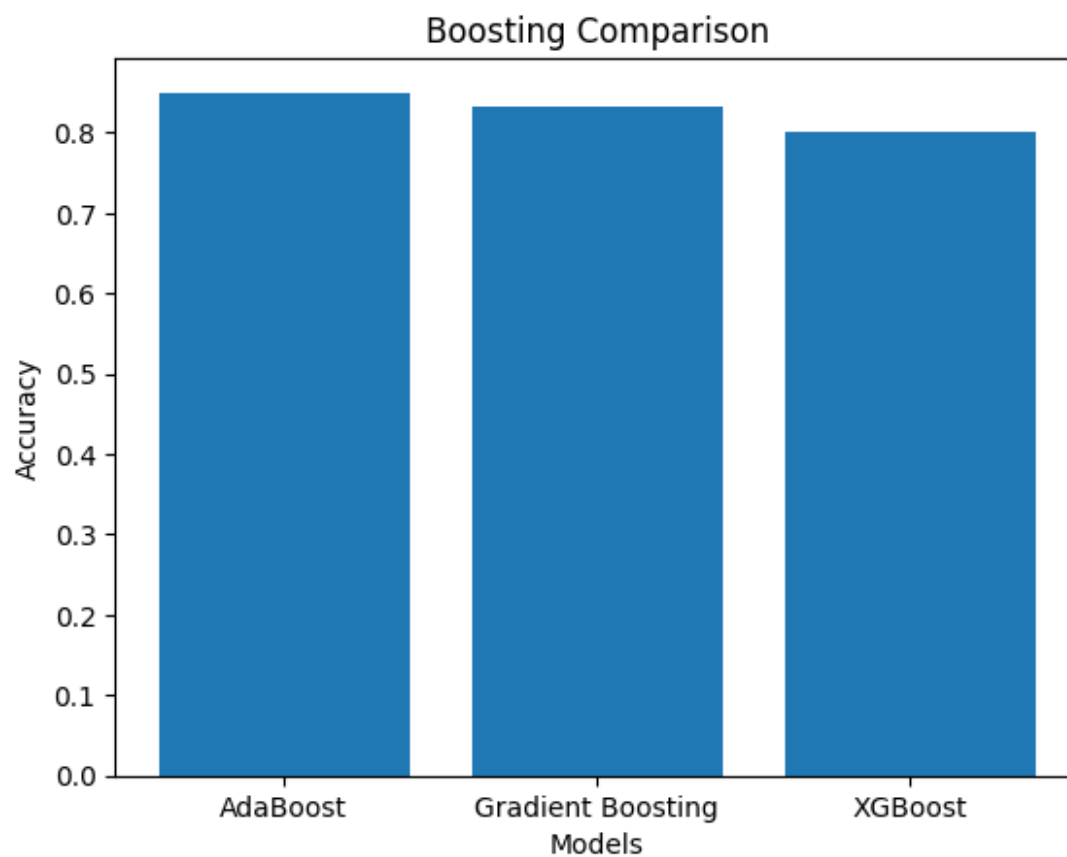
# Plot
models = ['AdaBoost', 'Gradient Boosting', 'XGBoost']
accuracies = [acc_ada, acc_gb, acc_xgb]
```

```
plt.bar(models, accuracies)
plt.title("Boosting Comparison")
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.show()
```

AdaBoost Accuracy: 0.85

Gradient Boosting Accuracy: 0.8333333333333334

XGBoost Accuracy: 0.8



Result:

Thus, the boosting algorithms AdaBoost, Gradient Boosting, and XGBoost were successfully implemented and compared based on their classification accuracy.

EXPERIMENT – 9

Register No.: 230701350

Name: SUDHARSHAN KRISHNAA L K

A PYTHON PROGRAM TO IMPLEMENT KNN AND K-MEANS ALGORITHMS

Aim

To implement K-Nearest Neighbors (KNN) classification and K-Means clustering algorithms using sample datasets.

Algorithm – KNN

1. Import the required libraries and define the training dataset.
2. Calculate the Euclidean distance between the test point and training points.
3. Sort the distances in ascending order.
4. Select the nearest k neighbors based on minimum distance.
5. Perform majority voting among the selected neighbors.
6. Display the predicted class label for the test point.

Algorithm – K-Means

1. Import the required libraries and define the input dataset.
2. Initialize the centroids for the required number of clusters.
3. Calculate the distance between each point and centroids.
4. Assign each data point to the nearest centroid.
5. Update the centroids by calculating the mean of cluster points.
6. Repeat the process until final clusters and centroids are obtained.

Code:

KNN

```
import math
from collections import Counter

# Function to calculate Euclidean distance
def euclidean_distance(p1, p2):
```

```

    sum = 0
    for i in range(len(p1)):
        sum += (p1[i] - p2[i]) ** 2
    return math.sqrt(sum)

# KNN function
def knn(train_data, train_labels, test_point, k):
    distances = []

    # Calculate distance from test point to all training data
    for i in range(len(train_data)):
        dist = euclidean_distance(train_data[i], test_point)
        distances.append((dist, train_labels[i]))

    # Sort distances
    distances.sort()

    # Get k nearest neighbors
    neighbors = distances[:k]

    # Extract labels of neighbors
    labels = [label for (_, label) in neighbors]

    # Majority voting
    most_common = Counter(labels).most_common(1)
    return most_common[0][0]

# Sample dataset
train_data = [
    [2, 4],
    [4, 6],
    [6, 8],
    [8, 10]
]

train_labels = ['A', 'A', 'B', 'B']

```

```
# Test point
test_point = [5, 7]

# Value of k
k = 3

# Prediction
result = knn(train_data, train_labels, test_point, k)
print("Test Point:", test_point)
print("Predicted Class:", result)
```

Test Point: [5, 7]

Predicted Class: A

K-MEANS

```
import math
```

```
# Function to calculate Euclidean distance
def distance(p1, p2):
    return math.sqrt(sum((p1[i] - p2[i])**2 for i in range(len(p1))))

# Function to calculate mean of points
def mean(points):
    n = len(points)
    return [sum(p[i] for p in points)/n for i in range(len(points[0]))]

# K-Means function
def kmeans(data, k, iterations=10):
    # Step 1: Initialize centroids (first k points)
    centroids = data[:k]

    for _ in range(iterations):
        clusters = [[] for _ in range(k)]

        # Step 2: Assign points to nearest centroid
        for point in data:
            distances = [distance(point, c) for c in centroids]
            min_index = distances.index(min(distances))
            clusters[min_index].append(point)
```

```

# Step 3: Update centroids
for i in range(k):
    if clusters[i]: # avoid empty cluster
        centroids[i] = mean(clusters[i])
return centroids, clusters

# Sample dataset
data = [
    [1, 2],
    [2, 3],
    [3, 4],
    [8, 9],
    [9, 10],
    [10, 11] ]
k = 2
centroids, clusters = kmeans(data, k)

# Output
print("Final Centroids:", centroids)
print("\nClusters:")
for i, cluster in enumerate(clusters):
    print(f"Cluster {i+1}: {cluster}")

```

Final Centroids: [[2.0, 3.0], [9.0, 10.0]]

Clusters:

Cluster 1: [[1, 2], [2, 3], [3, 4]]

Cluster 2: [[8, 9], [9, 10], [10, 11]]

Result:

Thus, the KNN classification and K-Means clustering algorithms were successfully implemented, and the data points were correctly classified and clustered.

EXPERIMENT – 10

Register No.: 230701350

Name: SUDHARSHAN KRISHNAA L K

A PYTHON PROGRAM TO IMPLEMENT PRINCIPAL COMPONENT ANALYSIS

Aim:

To implement Principal Component Analysis (PCA) for reducing the dimensionality of the given dataset.

Algorithm:

1. Import the required library and define the input dataset.
2. Calculate the mean of the dataset and center the data.
3. Compute the covariance matrix of the centered data.
4. Calculate the eigenvalues and eigenvectors of the covariance matrix.
5. Sort the eigenvectors based on descending eigenvalues.
6. Select the principal component and transform the data into reduced dimensions.

Code:

```
import numpy as np
# Step 1: Sample dataset (2D → reduce to 1D)
data = np.array([
    [2, 4],
    [4, 6],
    [6, 8],
    [8, 10]
])
# Step 2: Compute mean
mean = np.mean(data, axis=0)
# Step 3: Center the data
centered_data = data - mean
```

```

# Step 4: Compute covariance matrix
cov_matrix = np.cov(centered_data.T)
# Step 5: Compute eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

# Step 6: Sort eigenvectors by eigenvalues (descending)
sorted_indices = np.argsort(eigenvalues)[::-1]
eigenvectors = eigenvectors[:, sorted_indices]
# Step 7: Select top k eigenvectors (k = 1)
k = 1
principal_components = eigenvectors[:, :k]

# Step 8: Transform data
reduced_data = np.dot(centered_data, principal_components)
# Output
print("Original Data:\n", data)
print("\nReduced Data (1D):\n", reduced_data)

```

Original Data:

```

[[ 2  4]
 [ 4  6]
 [ 6  8]
 [ 8 10]]

```

Reduced Data (1D):

```

[[-4.24264069]
 [-1.41421356]
 [ 1.41421356]
 [ 4.24264069]]

```

Result:

Thus, Principal Component Analysis was successfully implemented, and the dimensionality of the dataset was reduced from 2D to 1D.